

# Reward Information Bandwidth for Code LLM Training: A Negative Result Establishing Scale Requirements

Anonymous

## Abstract

We investigate whether reward information bandwidth—the amount of feedback conveyed per gradient update—affects reinforcement learning convergence for code generation. We compare three reward functions: binary pass/fail (1 bit), continuous test pass rate ( $\sim 1.5$  bits), and high-bandwidth combining pass rate with categorical error type ( $\sim 2$  bits). At proof-of-concept scale (1 epoch, 3 seeds per condition on CodeLlama-7B), all conditions produce 0% pass@1, preventing meaningful comparison. This null result establishes a lower bound for our experimental setup: PoC-scale experiments with REINFORCE on this model/dataset combination are insufficient for reward ablation studies. We report this negative finding to provide scale guidance for future work and document the methodology for adequate-scale replication.

## 1 Introduction

Despite theoretical expectations that denser reward signals should accelerate reinforcement learning, we find that at proof-of-concept scale, all feedback granularities—binary, categorical, and high-bandwidth—produce identical outcomes: zero learning. This finding highlights the importance of adequate experimental scale for code LLM reward ablation studies.

Reinforcement learning from execution feedback (RLEF) has emerged as a promising paradigm for improving code LLMs. Prior work has explored various feedback granularities: binary pass/fail rewards Le et al. [2022], continuous test pass rates Liu et al. [2023], and multi-signal combinations incorporating error type information Shojaei et al. [2023]. Information theory suggests that higher bandwidth signals—those providing more bits per gradient update—should enable more precise credit assignment and faster convergence.

Liu et al. [2023] compared binary and fine-grained feedback in RLTF, finding improvements with richer signals. Our work aimed to provide additional controlled comparison using the “information bandwidth” framework. However, our PoC-scale experiment was insufficient to produce any learning signal.

**Important context:** We use REINFORCE with advantage baseline, not PPO. Prior successful work (CodeRL, PPOCoder) used different algorithms and trained for multiple epochs. Our failure may reflect algorithm choice, scale, or both.

The result was that at proof-of-concept scale (1 epoch, 3 seeds per condition), all nine training runs produced 0% pass@1 on the evaluation set. No condition learned. The statistical comparison we planned was impossible—there was no variance to analyze.

**Zero-shot baseline:** CodeLlama-7B-Instruct achieves approximately 30–40% pass@1 on MBPP in zero-shot evaluation Rozière et al. [2023]. Our 0% post-training result indicates either catastrophic forgetting during training or evaluation methodology differences. This is a critical limitation we did not fully diagnose.

This null result is not a falsification of the information bandwidth hypothesis. It is a finding about our specific experimental setup. Our PoC compute budget ( $\sim 94$  gradient steps per epoch with batch size 4) was below the threshold at which policy improvement emerges for this configuration.

We report this negative result because it documents:

1. **Setup-specific lower bound.** This specific configuration (REINFORCE, 1 epoch, CodeLlama-7B, MBPP) does not produce learning. We cannot generalize to all code LLM reward ablation without additional evidence.
2. **Methodology for replication.** Future researchers can attempt adequate-scale versions of this experiment with the information bandwidth framework.

3. **Diagnostic gap.** We lacked training curves, loss plots, and intermediate metrics that would distinguish “no learning” from “catastrophic forgetting” from “implementation error.”

## 2 Related Work

We review execution-based reward design for code LLMs and multi-granularity feedback approaches.

### 2.1 Execution Feedback for Code LLMs

CodeRL Le et al. [2022] established the RLEF paradigm using binary execution rewards. PPOCoder Shojaee et al. [2023] adapted Proximal Policy Optimization for code generation. Both works trained for multiple epochs and report final accuracy but not samples-to-threshold or learning curves.

### 2.2 Multi-Granularity Feedback

RLTF Liu et al. [2023] introduced fine-grained feedback incorporating test pass rates and error type information, comparing it to binary rewards. Their results suggested that richer feedback improves performance. Our work builds on RLTF’s multi-granularity concept but uses the “information bandwidth” framing and different infrastructure.

### 2.3 Process Supervision

Process reward models Lightman et al. [2023] demonstrated that step-by-step supervision outperforms outcome-only supervision for mathematical reasoning.

## 3 Methodology

We describe our approach to measuring reward information bandwidth effects on code LLM training.

### 3.1 Information Bandwidth Framework

We frame reward granularity in information-theoretic terms (approximate bit estimates):

**LOW (Binary):**  $r \in \{0, 1\}$  provides  $\sim 1$  bit per update.

**MEDIUM (Continuous):**  $r = n_{\text{passed}}/n_{\text{total}}$  provides additional information via pass rate granularity.

**HIGH (Categorical + Continuous):**  $r = 0.5 \times \text{categorical} + 0.3 \times \text{pass\_ratio} + 0.2 \times \text{partial\_credit}$  combines multiple signals.

*Note: Precise bit calculations would require entropy analysis of the actual reward distributions, which we did not perform. The “ $\sim 1.5$  bits” and “ $\sim 2$  bits” estimates are heuristic.*

### 3.2 Error-Type Scoring

Table 1: Error type scoring scheme

| Error Type        | Score | Rationale                  |
|-------------------|-------|----------------------------|
| Syntax error      | 0.00  | Code does not parse        |
| Runtime error     | 0.25  | Code runs but crashes      |
| Assertion failure | 0.50  | Code runs but wrong output |
| Pass              | 1.00  | Code is correct            |

### 3.3 Training Configuration

- **Model:** CodeLlama-7B-Instruct
- **Training data:** MBPP sanitized ( $\sim 374$  problems)
- **Evaluation:** MBPP validation (50 problems)
- **Algorithm:** REINFORCE with advantage baseline (not PPO)
- **Epochs:** 1 (PoC scale)
- **Seeds:** 3 per condition
- **Batch size:** 4
- **Gradient steps:**  $\sim 94$  per epoch

## 4 Experimental Setup

### Research Questions:

- RQ1: Does HIGH reach  $\text{pass@1} > 0.3$  faster than LOW?
- RQ2: Does MEDIUM outperform LOW?
- RQ3: Does error-type distribution differ?

### Evaluation:

- Primary:  $\text{pass@1}$  on validation subset
- Statistical: t-test, Cohen’s d,  $p < 0.05$

### Missing diagnostics (limitation):

- No training loss curves recorded
- No KL divergence or entropy monitoring
- No intermediate checkpoints
- No zero-shot baseline verification in our evaluation setup

## 5 Results

### 5.1 Main Results

All conditions failed to exceed floor performance:

| Table 2: Final pass@1 results across conditions |              |        |
|-------------------------------------------------|--------------|--------|
| Condition                                       | Final pass@1 | Status |
| LOW (Binary)                                    | 0.00%        | Floor  |
| MEDIUM (Continuous)                             | 0.00%        | Floor  |
| HIGH (Bandwidth)                                | 0.00%        | Floor  |

No condition reached the 30% threshold required for samples-to-threshold comparison.

## 5.2 Statistical Analysis

With all values at zero and no variance, statistical comparison was mathematically impossible:

- t-statistic: undefined (division by zero variance)
- p-value: undefined
- Cohen’s d: undefined

## 5.3 Missing Evidence

We cannot distinguish between:

1. Insufficient training scale (hypothesis)
2. Catastrophic forgetting (plausible given 0% vs  $\sim 30\%$  zero-shot)
3. Implementation error (not ruled out)
4. Algorithm unsuitability (REINFORCE vs PPO)

# 6 Discussion

## 6.1 Key Findings

1. **This configuration failed.**  $\sim 94$  gradient steps with REINFORCE on CodeLlama-7B/MBPP produced no learning.
2. **Hypothesis remains untested.** Null result  $\neq$  falsification.
3. **Critical diagnostic gap.** Without training curves, we cannot determine failure mode.

## 6.2 Limitations

**Decisive:**

- 1 epoch: insufficient gradient updates ( $\sim 94$  steps)
- REINFORCE algorithm: high variance, may require more samples than PPO
- No training diagnostics: cannot verify policy updates occurred
- No zero-shot baseline in our setup: cannot rule out catastrophic forgetting

**Acknowledged:**

- 3 seeds: insufficient for variance estimation
- 0.5/0.5 weighting: heuristic, not tested
- Bit estimates: approximate, not rigorously calculated
- Single model size (7B)
- No hyperparameter search

### 6.3 What Would Make This Conclusive

A valid test would require:

- $\geq 3$  epochs or until validation plateau
- PPO or lower-variance algorithm
- Training curves showing policy improvement (or lack thereof)
- Zero-shot baseline verified in same evaluation setup
- $\geq 5$  seeds per condition

## 7 Conclusion

We attempted to test whether information bandwidth in reward functions affects code LLM training convergence. At PoC scale with REINFORCE, all conditions produced 0% pass@1, preventing any comparison.

This is a documentation of experimental failure, not a research contribution. We cannot claim to have established minimum scale requirements because we lack the diagnostics to determine why our experiment failed.

The hypothesis remains open. Future work should use adequate scale, include training diagnostics, and verify the experimental setup produces learning before attempting reward ablation.

## References

- Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven C. H. Hoi. Coderl: Mastering code generation through pretrained models and deep reinforcement learning. *Advances in Neural Information Processing Systems*, 35, 2022.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. *arXiv preprint arXiv:2305.20050*, 2023.
- Jiate Liu, Yiqin Xia, Yongjiang Wang, Yue Zhang, Chao Wang, Ying Ding, and Junlan Xu. Rltf: Reinforcement learning from unit test feedback. *arXiv preprint arXiv:2307.04349*, 2023.
- Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.
- Parshin Shojaei, Aneesh Jain, Patrick Fernandes, Ethan Perez, Jiaheng Mao, Hui Xiong, and Graham Neubig. Execution-based code generation using deep reinforcement learning. *arXiv preprint arXiv:2306.05826*, 2023.